

Progress reports show what happened. I built what comes next.

A portfolio case study showing how I turned a real construction progress workbook into a Python-based early warning system. The pipeline reshaped planned and actual progress data, engineered forecasting features, compared models correctly through time, and produced package-level next-update predictions.

Python

Pandas

Scikit-learn

Feature Engineering

Ridge Regression

Random Forest

0.994

R2 ON WEEKLY TEST DATA

0.012

MEAN ABSOLUTE ERROR

5

FEATURE FAMILIES

Ridge

BEST MODEL

WHAT THIS PROVES

Not just dashboard building. This project shows the full analytical delivery cycle: messy source data, structured transformation, feature design, honest model comparison, readable outputs and a practical route into project-controls decision making.

By the time the report shows a problem, the team is already reacting.

Project-controls teams usually have planned and actual progress data. The issue is timing. The report tells you a work package is behind after the update has already landed. That is useful, but it is reactive.

QUESTION

Can planned and actual progress data that already exists in project reporting be reshaped into a signal that predicts the next reporting outcome at package level?

INPUT

Real operational progress workbook

TARGET

Next-step progress variance

OUTPUT

Package-level early warning signal

I used progress variance as the forecasting target: actual cumulative progress minus planned cumulative progress. A negative value means the package is behind. The model does not just calculate today's variance. It estimates where the next update is likely to land.

I did not start with a clean dataset. I started with a reporting workbook.

The original workbook was structured for people, not models. Dates ran across columns. Planned and actual progress lived in separate sheets. That works for manual monitoring, but it is the wrong shape for forecasting.

Activity ID	Activity Name	Budgeted Units	Actual Units	At Completion Units	Spreadsheet Field	11-Feb-25	12-Feb-25	13-Feb-25	14-Feb-25	15-Feb-25	16-Feb-25	17-Feb-25	18-Feb-25	19-Feb-25	20-Feb-25
1	Development and Upgrading CR	1555811	0	1555811	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
2	Installation & Performance SDC	1555811	0	1555811	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
3	Installation	1555811	0	1555811	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
4	Design/Construct/Install	1555811	0	1555811	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
5	Design/Construct/Install	1555811	0	1555811	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
6	Design/Construct/Install	1555811	0	1555811	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
7	Design/Construct/Install	1555811	0	1555811	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
8	Design/Construct/Install	1555811	0	1555811	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
9	Design/Construct/Install	1555811	0	1555811	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
10	Design/Construct/Install	1555811	0	1555811	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
11	Design/Construct/Install	1555811	0	1555811	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
12	Design/Construct/Install	1555811	0	1555811	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
13	Design/Construct/Install	1555811	0	1555811	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
14	Design/Construct/Install	1555811	0	1555811	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
15	Design/Construct/Install	1555811	0	1555811	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
16	Design/Construct/Install	1555811	0	1555811	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
17	Design/Construct/Install	1555811	0	1555811	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
18	Design/Construct/Install	1555811	0	1555811	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
19	Design/Construct/Install	1555811	0	1555811	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
20	Design/Construct/Install	1555811	0	1555811	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
21	Design/Construct/Install	1555811	0	1555811	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
22	Design/Construct/Install	1555811	0	1555811	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
23	Design/Construct/Install	1555811	0	1555811	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
24	Design/Construct/Install	1555811	0	1555811	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
25	Design/Construct/Install	1555811	0	1555811	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
26	Design/Construct/Install	1555811	0	1555811	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
27	Design/Construct/Install	1555811	0	1555811	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
28	Design/Construct/Install	1555811	0	1555811	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
29	Design/Construct/Install	1555811	0	1555811	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
30	Design/Construct/Install	1555811	0	1555811	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%

Figure A.2: Existing Workflow of Cumulative Planned Progress

Shows the original spreadsheet-based reporting format used to compare cumulative planned and cumulative actual progress over time, together with interval progress values. This table reflects the baseline-versus-update logic that was later reproduced programmatically in the Python pipeline.

FIG 01 ORIGINAL SPREADSHEET-BASED PROGRESS REPORTING

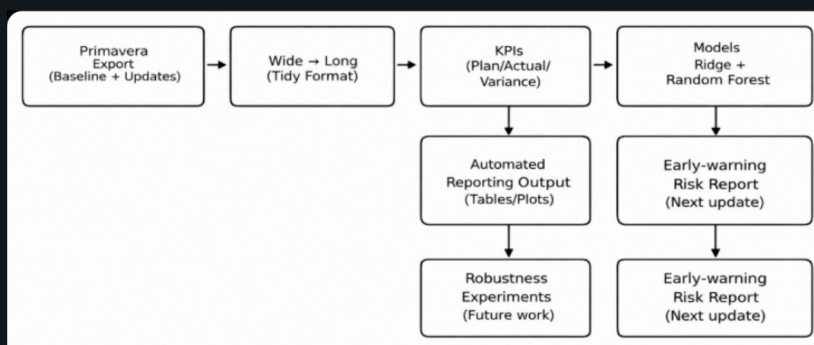


FIG 02 END-TO-END PYTHON PIPELINE

The main work was turning reporting data into modelling data.

This was the development cycle: reshape the workbook, align planned and actual values, construct the variance KPI, then turn the next reporting step into the prediction target.

01

Wide-to-long conversion

Dates were unpivoted so time became a proper variable instead of a set of spreadsheet columns.

02

Merge and variance construction

Planned and actual values were aligned by package and date, then converted into a progress variance KPI.

03

Daily and weekly datasets

I tested different time granularities to see whether reporting frequency affected model performance.

04

Next-step target creation

Variance was shifted forward within each package, turning the workbook into a supervised forecasting problem.

The features were designed around how progress actually moves.

The model needed more than the current position of a package. It needed recent history, pace of movement and whether deviation from plan was stable or getting worse.

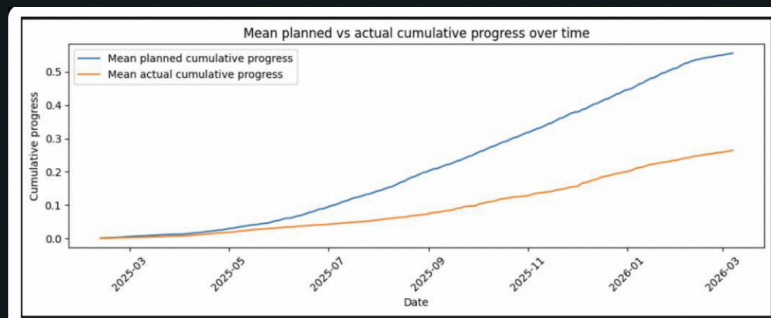


FIG 03 PLANNED VERSUS ACTUAL PROGRESS GAP

CURRENT STATE

Actual progress, planned progress and current variance set the baseline position.

LAG FEATURES

Previous reporting values gave the model memory of package behaviour.

ROLLING FEATURES

Short-window averages and standard deviations captured trend and instability.

INTERVAL FEATURES

Change between updates captured pace rather than just position.

TIME INDEX

The sequence of updates gave the model a sense of project lifecycle timing.

I started with Ridge Regression deliberately, not because it was the simpler option.

The engineered features were related. Current variance, lagged variance, rolling variance and interval variance all measure overlapping parts of the same behaviour. Ridge is useful here because it keeps all features active while shrinking unstable coefficients.

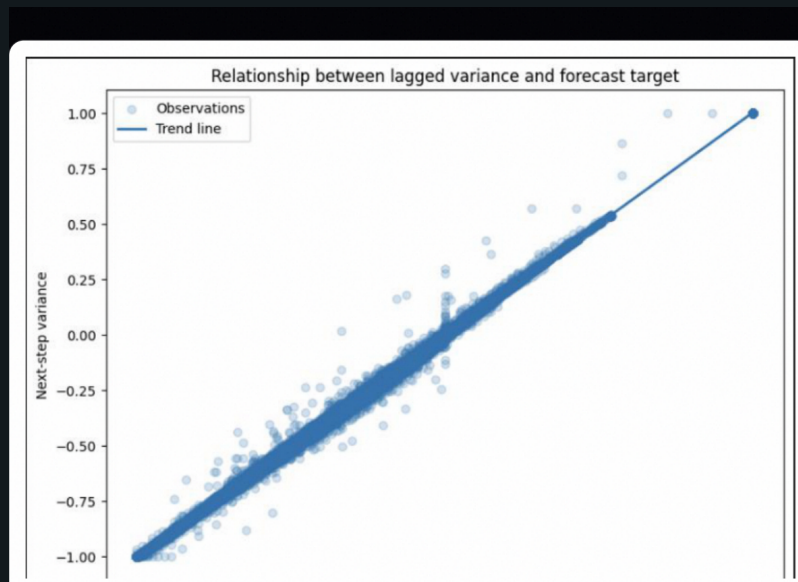


FIG 04 LAGGED VARIANCE WAS THE STRONGEST INDIVIDUAL SIGNAL

EVALUATION PROTOCOL

I used chronological train-test splits. For forecasting, shuffling records is not acceptable because it leaks future information into training and produces metrics that look stronger than they are.

Ridge did not just win. It was the right tool for this data.

The weekly dataset produced the clearest result. Ridge achieved stronger accuracy than every Random Forest configuration while using a model that was easier to explain.

WEEKLY MODEL COMPARISON

Model	MAE	RMSE	R2	Verdict
Ridge	0.011809	0.029473	0.993952	Best overall
RF Deep	0.024842	0.065241	0.970365	Best RF
RF Default	0.025118	0.065962	0.969707	Close to deep
RF Simple	0.027664	0.068066	0.967743	Weakest

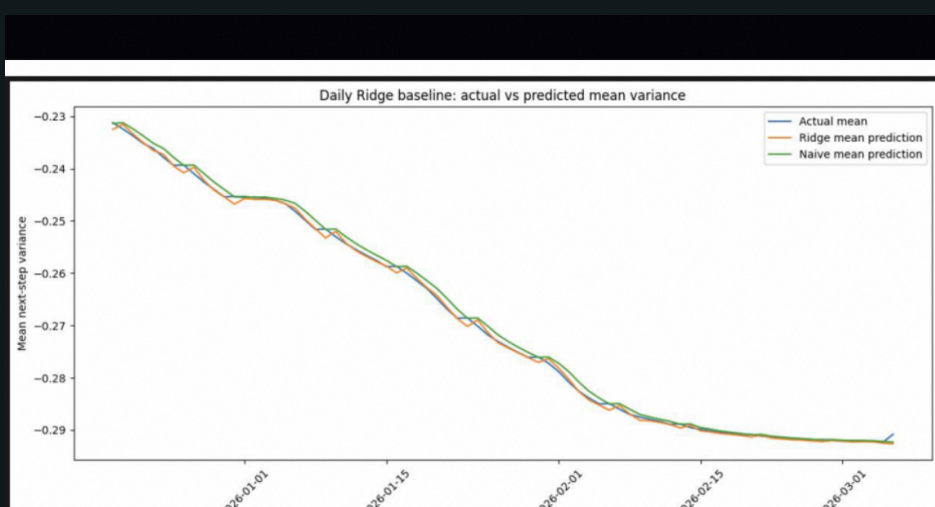


FIG 05 ACTUAL VERSUS PREDICTED MEAN VARIANCE

The result was supported by readable model evidence.

I did not want a black-box result that only looked good in a metrics table. The evidence needed to explain why Ridge worked and why Random Forest underperformed.

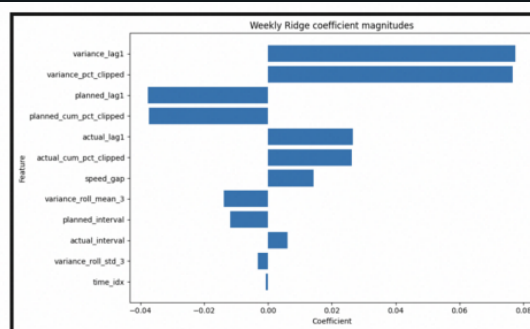


Figure 5.7. Weekly Ridge coefficient magnitudes

FIG 06 RIDGE COEFFICIENT MAGNITUDES

INTERPRETATION

Variance lag and current variance carried the strongest signal, but Ridge still distributed weight across several features instead of depending on one variable.

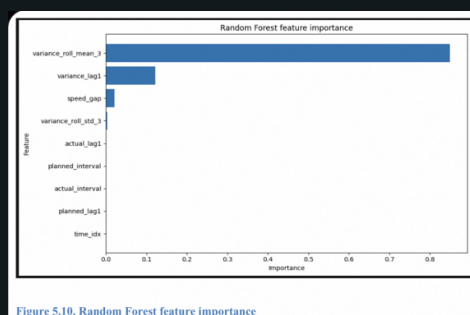


Figure 5.10. Random Forest feature importance

FIG 07 RANDOM FOREST FEATURE IMPORTANCE

The pipeline produced forecasts at the level where decisions happen.

The aim was not only a portfolio-level average. Project teams manage risk through work packages, so the forecast output had to work at package level.

Appendix C. Package-Level prediction and diagnostic evidence

This appendix provides an illustrative package-level example from the weekly test set to show that the implemented forecasting pipeline produces predictions for individual packages rather than only aggregate summaries.

Row ID	Package	Date	Actual next-step variance	Ridge prediction	RF prediction	Naive prediction	Ridge abs. error	RF abs. error	Naive abs. error
0	2. Development and Upgrading Of Roads and Infrast...	2025-12-23	-0.256625	-0.252976	-0.260681	-0.243006	0.003649	0.004056	0.013619
1	2. Development and Upgrading Of Roads and Infrast...	2025-12-30	-0.262643	-0.262778	-0.266119	-0.248428	0.000135	0.003476	0.014215
2	2. Development and Upgrading Of Roads and Infrast...	2026-01-06	-0.268715	-0.267191	-0.268585	-0.256625	0.001524	0.000130	0.012090
3	2. Development and Upgrading Of Roads and Infrast...	2026-01-13	-0.274769	-0.273640	-0.271634	-0.262643	0.001129	0.003135	0.012126
4	2. Development and Upgrading Of Roads and Infrast...	2026-01-20	-0.283815	-0.279210	-0.273364	-0.268715	0.004606	0.010451	0.015100
5	2. Development and Upgrading Of Roads and Infrast...	2026-01-27	-0.290952	-0.289552	-0.295099	-0.274769	0.001400	0.004147	0.016183
6	2. Development and Upgrading Of Roads and Infrast...	2026-02-03	-0.297756	-0.295694	-0.299373	-0.283815	0.002063	0.001617	0.013941
7	2. Development and Upgrading Of Roads and Infrast...	2026-02-10	-0.302117	-0.301877	-0.300467	-0.290952	0.000340	0.001650	0.011165
8	2. Development and Upgrading Of Roads and Infrast...	2026-02-17	-0.304205	-0.303904	-0.300791	-0.297756	0.000301	0.003414	0.006449
9	2. Development and Upgrading Of Roads and Infrast...	2026-02-24	-0.305226	-0.303832	-0.301631	-0.302117	0.001394	0.002595	0.003109

Figure C.1: Detailed weekly package-level prediction comparison for the selected package “Development and Upgrading Of Roads and Infrastructure In The Mafrag Industrial City”

FIG 08 PACKAGE-LEVEL PREDICTION OUTPUT

WHY INTERPRETABILITY MATTERED

A model nobody trusts gets ignored, even if the test score is strong. Ridge gave a result I could explain: package variance tends to carry forward, related variance features reinforce that signal, and the model remains readable enough for a real project-controls environment.

RANDOM FOREST DIAGNOSIS

Random Forest concentrated too much importance on the rolling variance mean. That helped explain why a more flexible model still underperformed the regularised linear approach.

This project sits beyond dashboards.

The value is not only the model score. The value is the judgement across the whole development cycle: reshaping messy operational data, engineering features that reflect the domain, comparing models without data leakage, and explaining the output in a way project teams can use.

01 END-TO-END PYTHON PIPELINE

Ingestion, cleaning, reshaping, feature engineering, model training, evaluation and output generation.

02 DOMAIN-BASED FEATURE ENGINEERING

Five feature families designed around how construction progress behaves over time.

03 RIGOROUS MODEL COMPARISON

Chronological test splits and multiple model configurations, not a decorative benchmark.

04 PACKAGE-LEVEL OUTPUTS

Predictions created where project teams make decisions, not only at portfolio-average level.

05 INTERPRETABLE MODEL CHOICE

Ridge was selected because it matched the data structure, not because it sounded impressive.

PRACTICAL NEXT STEP

Next step: connect package forecasts to critical path and float logic, so warnings are prioritised by schedule risk rather than reviewed uniformly.